MODULAR REFERENTIAL SCRIPTURE: NEEDLE & STEELSSYSTEM CODEX

Semantic-Lossless Quantized Architecture (LQ-1) · Cactus Needle 3 / 2 Edge Runtime · Neurosymbolic Generative Passage

SOURCE: AbuZar-Ansarii/Needle & kamiltrefon69/Needle (@374eaaf) · TARGET: aistudio.google.com · FORMAT: LQ-1 / QSS-U v3.0

1. Executive Synthesis & Master QSS-U Register Vector

[MOD-01 · CORE-VECTOR]

This blueprint codifies the verified architecture of **AbuZar-Ansarii/Needle** (synchronized with fork **kamiltrefon69/Needle** at commit `374eaaf`) and integrates it with the **SteelsSystem Algorithm Codex** and the **Hybrid Neurosymbolic "Generative Passage" (Candidate C)** framework. Designed as a modular scripture for direct ingestion into **Google AI Studio (aistudio.google.com)**, each page functions as a self-contained, addressable reference block (`[MOD-01]` through `[MOD-06]`) for System Instructions, Context Caching, Function Calling declarations, and Structured Outputs.

[Qntz:QSS-U Master Execution Vector — Copy Directly into AI Studio System Instructions]

```
[Qntz:QSS-U|L=EN|P=Lead_Edge_AI_Systems_Architect|T=On_Device_Termux_Agentic_Runtime|W4=
(WHAT:Implement_Needle_Termux_Hardware_Agent,WHEN:Autonomous_repo_intake_and_fork_sync,WHY:Ze
ro_cloud_14MB_29MB_local_phone_control_RAG_extraction,HOW:Sanitize_PRoot_bind_Cactus_runtime_
register_Termux_API_tools_expose_Flask_CLI_Telegram)|HYP=Strict_grammar_constrained_tool_cont
racts|OBS=
(A:PRoot_Ubuntu_ARM64_sandbox,B:Hardware_timeout_15s_fallback_simulator,C:Zero_cloud_LLM_infe
rence_local_SQLite_vault)|M=OP|V=3.0|FLOW=SanitizeEnv->LoadCactusEngine->RegisterTermuxTools-
>ExecuteGrammarLoop-
>ExposeMultiModalTransport|LOAD=Cactus_Needle_Weights+Termux_API_Bridge+Pydantic_Extraction_S
chemas|MM=0|MAC=
[CactusRuntimeEngine,TermuxHardwareBridge,KnowledgeVaultRAG,MultiModalExtractorTriage,OmniCha
nnelServer]|R=Zero_cloud_inference_calls+100%_grammar_valid_tool_dispatch+Desktop_simulator_f
allback|NEXT=Initialize_PRoot_env_and_bind_needle_model|STOP=Missing_cactus_needle_runtime_or
_unhandled_PRoot_Bionic_env_collision]
```



Figure 1.1 — Deterministic 5-Stage QSS-U Execution Flow and Binary Guard Predicate (State A Halt vs. State B Execution).

2. Google AI Studio Direct Ingestion & Modular Reference Protocol

[AISTUDIO-PROTOCOL]

- **System Instruction Anchoring:** Upload this PDF directly into the Google AI Studio prompt context or Context Cache and reference modules by tag: `[MOD-01]` (QSS-U Vector & Flow), `[MOD-02]` (Cactus Models & PRoot Bootstrap), `[MOD-03]` (Termux 5-Domain Tool Registry), `[MOD-04]` (SteelsSystem Alg 01–08), `[MOD-05]` (Neurosymbolic Dual-Channel Voice/State), and `[MOD-06]` (LQ-1 JSON & Rust Cryptographic Guard).
- **Recommended AI Studio Run Settings:** Set `Temperature = 0.15` for deterministic tool-schema synthesis and `Qntz:QSS-U` compilation, or `Temperature = 0.40` when executing `[MOD-05]` voice-to-voice biocomputer co-exploration. Enable **Structured Outputs** when targeting Channel 2 scratchpad JSON or Pydantic v2 extraction schemas.
- **Zero-Mockup Contract Enforcement (`MM=0`):** When instructing Gemini in AI Studio to extend `app.py` or `termux_needle.py`, mandate preservation of `IS_TERMUX` detection, 15-second `run_cmd(args)` timeouts, and deterministic `[Simulated ...]` desktop fallbacks.

3. Foundation Models, Runtime Stack & PRoot Sanitization [MOD-02 · RUNTIME-CORE]

The runtime engine is powered by **Cactus Compute's `cactus-needle>=3.0.0`** package, pairing a prebuilt sub-1 MB native execution binary with ultra-compact `.cact` weight artifacts. Users can hot-swap between **Needle 3** and **Needle 2** on the fly via the Web UI header toggle (`127.0.0.1:5000`), CLI commands (`/model 2`, `/model 3`), or Telegram bot commands without restarting the Python process.

Component / Model	Architecture & Size	Quantization / Footprint	Core Operational Capabilities in Needle
`Needle 3` (`Default Engine`)	20-layer Laddered Simple Attention Network (121M params; sliceable 2L–20L)	CQ2-bit (Cactus Quants) `needle3.cact` (~29 MB; 8–29 MB slice range; ≥4 GB RAM)	Byte-level grammar-constrained tool calling (`@needle.tool`), Pydantic v2 structured extraction, 3,072-dim native embeddings, calibrated confidence head (`0.1` floor).
`Needle 2` (`Legacy Fast Core`)	Simple Attention Network (SAN) distilled from Gemini 3.1 (26M params)	`needle2.cact` (14 MB binary artifact)	Ultra-fast single/multi-call function routing (6,000 tok/s prefill, 1,200 tok/s decode). Invoked via `needle.Needle(tools=[...], generation=2)`.
`Whistle STT` (`Audio Companion`)	Shared-container speech-to-text acoustic model	`whistle.cact` (16.9 MB binary artifact)	7-language offline speech-to-text co-execution alongside `needle3.cact`; single-call audio-to-transcript-and-tool-calls JSON output.
`Embedding Head` (`/api/embed`)	Needle 3 native high-dimensional projection head	3,072-dimensional float vector output	100% offline semantic indexing for the SQLite Knowledge Vault RAG and natural-language contact book lookup.
`PRoot Sanitizer` (`SanitizeEnv`)	POSIX / ARM64 environment interceptor	Zero-overhead startup dictionary purge	Strips inherited `ANDROID_ROOT` and `ANDROID_DATA` inside PRoot Ubuntu so pip/runtime loads `manylinux2014_aarch64` instead of Android Bionic libc.
`Web & WSGI Host` (`app.py`)	`flask>=3.0.0` + `waitress>=3.0.0`	Localhost loopback `127.0.0.1:5000`	Serves responsive light/dark Web UI, live execution logs, active tool cancellation, model toggle, and 4 multimodal studio sidebars.
`Remote & CLI` (`termux_needle.py`)	Interactive CLI REPL + `pyTelegramBotAPI>=4.12.0`	Daemon background thread (`--telegram TOKEN`)	Allows full offline-phone hardware control from terminal or remote Telegram chat while LLM inference remains 100% on-device.

4. PRoot Bionic-to-Glibc Sanitization & Hardware Subprocess Pattern [BOOTSTRAP-CODE]

Inside Termux PRoot-Distro Ubuntu ARM64, Android leaks `ANDROID_ROOT` and `ANDROID_DATA` into the guest shell, causing Python wheel resolvers and native binaries to misidentify the environment as Bionic libc rather than standard glibc (`manylinux2014_aarch64`). Both `app.py` and `termux_needle.py` execute an immediate pre-import sanitization routine, bind `/data/data/com.termux/files/usr/bin` to PATH, and wrap all hardware calls in a 15-second timeout guard:

```
# Canonical Bootstrap, Environment Sanitizer & Grammar-Constrained Tool Pattern (app.py / termux_needle.py) import os, subprocess for _var in ("ANDROID_ROOT", "ANDROID_DATA"): os.environ.pop(_var, None) # Force manylinux2014_aarch64 resolution in PRoot Ubuntu TERMUX_BIN = "/data/data/com.termux/files/usr/bin" if os.path.exists(TERMUX_BIN) and TERMUX_BIN not in os.environ.get("PATH", ""): os.environ["PATH"] = f"{TERMUX_BIN}: {os.environ.get('PATH', '')}" IS_TERMUX = os.path.exists("/data/data/com.termux") import needle from pydantic import BaseModel def run_cmd(args: list[str], sim_tag: str = "Simulated Command") -> str: if not IS_TERMUX: return f"[{sim_tag}] Executed offline fallback for: {' '.join(args)}" try: res = subprocess.run(args, capture_output=True, text=True, timeout=15) return res.stdout.strip() or res.stderr.strip() or "OK" except (FileNotFoundError, PermissionError, subprocess.TimeoutExpired) as exc: return f"[{sim_tag} Fallback
```

5. Termux Hardware Bridge & 5-Domain Tool Grammar Registry [MOD-03 · HARDWARE-TOOLS]

The `TermuxHardwareBridge` exposes over 35 native Android hardware, sensor, radio, audio, and messaging capabilities by bridging Python `@needle.tool` functions to the **Termux:API** companion package. Every tool is governed by the 15-second hardware warm-up timeout and falls back cleanly on desktop environments.

Domain ID	Registered @needle.tool Functions	Underlying Termux:API Binaries	Operational Contract & Fallback Envelope
DOM-01 Hardware & Sensors	`toggle_flashlight`, `get_battery_status`, `set_brightness`, `vibrate_phone`, `read_sensors`, `take_screenshot`, `set_wallpaper`, `wake_lock`, `press_key`, `authenticate_fingerprint`	`termux-torch`, `termux-battery-status`, `termux-brightness`, `termux-vibrate`, `termux-sensor`, `termux-screencap`, `termux-wallpaper`, `termux-wake-lock`, `input keyevent`, `termux-fingerprint`	Reads JSON telemetry (battery %, health, temperature, accelerometer, light sensor) and mutates physical states. Biometric gate returns verified auth status or `[Simulated Biometric]`.
DOM-02 Connectivity & Radio	`get_wifi_info`, `scan_wifi`, `toggle_wifi`, `get_sim_info`, `get_cell_info`, `scan_nfc`, `get_gps_location`	`termux-wifi-connectioninfo`, `termux-wifi-scaninfo`, `termux-wifi-enable`, `termux-telephony-deviceinfo`, `termux-telephony-cellinfo`, `termux-nfc`, `termux-location`	Queries SSID/BSSID/IP, nearby Wi-Fi APs, cellular tower signal dBm, NFC tags, and GPS coordinates (lat/lon/alt). GPS warm-up protected by 15s subprocess cap.
DOM-03 Audio, Speech & Media	`speak_tts`, `set_volume`, `media_control`, `record_microphone`, `get_audio_info`	`termux-tts-speak`, `termux-volume`, `termux-media-player`, `termux-microphone-record`, `termux-audio-info`	Powers `JarvisVoiceLoop` spoken output, adjusts stream volumes, controls media playback (play/pause/stop), records `.3gp` audio, and checks Bluetooth routing.
DOM-04 Messaging, Calls & Contacts	`send_sms`, `list_sms`, `make_call`, `get_call_log`, `search_contacts`, `show_toast`, `show_notification`, `confirm_dialog`, `set_clipboard`, `get_clipboard`	`termux-sms-send`, `termux-sms-list`, `termux-telephony-call`, `termux-call-log`, `termux-contact-list`, `termux-toast`, `termux-notification`, `termux-dialog`, `termux-clipboard-set/get`	Executes outbound SMS/calls, reads SMS inbox for `SmartSMSTriage`, performs semantic contact lookup, posts notifications, and writes extracted OTP codes to clipboard.
DOM-05 Apps & System Actions	`open_app`, `download_file`, `share_content`	`am start / monkey (WhatsApp, YouTube, Chrome, Maps)`, `termux-download`, `termux-share`	Launches native Android package intents, queues system download manager transfers, and invokes the native Android share sheet.

6. Multimodal Intelligence Studios (Commit 374eaaf Additions) [MULTIMODAL-STUDIOS]

- **Offline "Jarvis" Voice Loop (`JarvisVoiceLoop`):** Combines hands-free offline speech recognition (Android STT / `whistle.cact` 16.9 MB companion) with Needle grammar-constrained tool routing and `termux-tts-speak` vocal synthesis, enabling completely screen-free hardware control and conversational interaction.
- **Local Knowledge Vault RAG (`KnowledgeVaultRAG`):** Embeds personal notes, credentials, and long-term memories into a local **SQLite3** database using Needle 3's native **3,072-dimensional embedding head (`/api/embed`)**. Performs exact cosine similarity retrieval across vault entries and the device contact book with zero network egress.
- **Offline Camera Document Extractor (`CameraDocumentExtractor`):** Captures document frames via the device camera and applies Needle 3's byte-level Pydantic v2 schema extraction to parse *Receipts*, *Invoices*, *Event Badges*, and *Business Cards* into strictly typed JSON structures offline.
- **Smart SMS & Notification Triage (`SmartSMSTriage`):** Ingests incoming SMS payloads via `termux-sms-list`, classifies messages offline into four canonical buckets (`OTP`, `Bank Transactions`, `Spam`, `Personal`), and automatically extracts one-time verification codes directly into the Android clipboard via `termux-clipboard-

7. SteelsSystem Algorithm Codex (Alg 01–08) Operational Mapping

`[MOD-04 · STEELSSYSTEM-ALGS]`

To unify formal defeasible reasoning with on-device edge execution, the eight core algorithms of the **SteelsSystem Algorithm Codex (`SteelsSystem.github.io`)** are mapped directly onto the runtime primitives of `AbuZar-Ansarii/Needle` (`kamiltrefon69/Needle`). Use this matrix in Google AI Studio when synthesizing formal verification guards around tool calls.

Alg ID	SteelsSystem Formal Module	Formal Invariant / Mechanism	Exact Instantiation in Needle (app.py / termux_needle.py)
`Alg 01`	**QSS-U Shorthand Parser & State Machine**	Pipe-delimited tuple AST with deterministic NEXT and STOP predicates	Enforces 5-stage FLOW (`SanitizeEnv`→`LoadCactusEngine`→`RegisterTermuxTools`→`ExecuteGramma` Triggers **State A Halt** on Bionic wheel collision or missing `.cact` weights.
`Alg 02`	**Carneades Argument Evaluation (CAES)**	Tripartite evaluation: *Ordinary Premises*, *Assumptions*, *Exceptions*	*Premises:* Grammar-valid tool args. *Assumptions:* `IS_TERMUX == True` & Android pern `TimeoutExpired` (>15s) or `PermissionError` routing cleanly to Simulator fallback.
`Alg 03`	**Crucibilum Burden-of-Proof Engine**	Calibrated confidence score $c \in [0, 1]$ evaluated against proof thresholds	Maps Needle 3's confidence head: $c < 0.10$ routed to `suppressed_calls` (empty list `[]` confirmation; $c \geq 0.75$ executes autonomously.
`Alg 04`	**SKSS/CLSL Drift & Tabularium Remapper**	Cosine similarity manifold scan + regex anchor locking	Uses Needle 3's 3,072-dim `/api/embed` vectors in SQLite `KnowledgeVaultRAG` to alig preventing intent drift via deterministic regex triggers.
`Alg 05`	**Ω > Δ > ◊ Axiomatic Moral & Safety Override**	Strict lexicographic dominance: Tier 1 (Ω) > Tier 2 (Δ) > Tier 3 (◊)	**Ω (Sovereignty):** 100% air-gapped inference + `termux-fingerprint` gate. **Δ (Mutating** schema validation. **◊ (Ambient):** Telemetry reads execute zero-friction.
`Alg 06`	**Liquid Certainty Thermodynamic Converter**	Compute-to-latency equilibrium via dynamic model depth	Hot-swaps between 20-layer 121M **Needle 3** (29 MB, deep extraction/RAG) and 26M **Nee** tok/s decode) or slices Laddered SAN depth (2L–20L).
`Alg 07`	**Armamentarium Walton Scheme Dissector**	Critical Question (CQ_k) verification on untrusted inputs	Audits incoming payloads in `SmartSMSTriage` (separating legitimate OTP/Bank from S Pydantic completeness in `CameraDocumentExtractor`.
`Alg 08`	**VaporSphere CIELAB & Lilly Inversion**	High-contrast telemetry manifold & topological state display	Renders live execution logs, active tool cancellation controls, and 3,072-dim vault search dashboard at `http://127.0.0.1:5000`.

9. Hybrid Neurosymbolic "Generative Passage" & Dual-Channel Interlock

`[MOD-05 · NEUROSYMBOLIC-VOICE]`

This module bridges `Needle`'s offline **Jarvis Voice Loop** and **Knowledge Vault RAG** with the **Hybrid Neurosymbolic "Generative Passage" Architecture (Candidate C)**. It couples Dr. John C. Lilly's 11-Level Human Biocomputer model (Levels 1–4 Hardware/Brain; Levels 5–10 Software/Metaprograms/Executive "I"; Level 11 Unknown) with Hierarchical Active Inference (VFE, EFE, habit prior $E(\pi)$, policy precision γ, preference precision γ_C, epistemic depth Φ, hyper-free energy F_{hyper}), ACT-R 4-question retrieval gating, and 3C knowledge calibration across two strictly partitioned runtime channels.

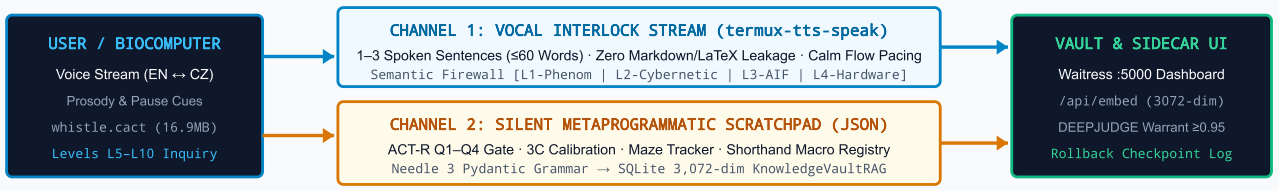


Figure 5.1 — Partitioned Dual-Channel Architecture Coupling Jarvis Voice Loop (Channel 1) with Silent SQLite RAG Scratchpad (Channel 2).

10. Channel 2 Silent State & Shorthand Macro Schema (Pydantic / SQLite Vault)

`[CHANNEL-2-SCHEMA]`

```
# Copy-Ready Channel 2 JSON State Skeleton (Updated Silently Per Turn via Needle 3 Grammar / AI Studio Structured Output) { "session_metadata": { "turn_index": 1, "active_language": "EN | CZ", "transport_mode": "PURE_AUDIO_LATENT | SIDECAR_SPA_WEBHOOK" }, "session_state": { "active_biocomputer_levels": ["L6_Program", "L7_Metaprogram", "L8_SelfMetaprogrammer"], "active_mode": "MAZE_NAVIGATION | TERMINOLOGY_AUDIT | SHORTHAND_CAPTURE | RELEVANCE_GATE", "act_r_buffer": { "current_goal_scheme": "<active exploration vector>", "retrieved_chunk_id": "<SQLite 3072-dim vault chunk ID>", "triage_classification": "ON_PATH | TANGENT_VALUABLE_PARKED | DRIFT_PRUNED", "relevance_justification": { "q1_store": "...", "q2_retrieve": "...", "q3_spread_match": "...", "q4_next_op": "..." } }, "metareasoning_controller": { "variational_free_energy_vfe": "LOW | MEDIUM | HIGH_SURPRISE", "policy_precision_gamma": 0.78, "preference_precision_gamma_C": 0.65, "recursion_depth_counter": 1, "cognitive_effort_divergence": "OPTIMAL_FLOW | UNDERLOAD | OVERLOAD_RISK" }, "maze_topology_tracker": { "active_stage": "STAGE_1_IMPASSE | STAGE_2_HALT_RECALL | STAGE_3_LOCK_OPACIFIED | STAGE_4_INVERTED_BALLOON | STAGE_5_CONSOLIDATED", "detected_blockade_device": "NONE | EVASION | DENIAL | REPRESSION_COEX | HABIT_PRIOR_LOCK", "lock_opacity_state": "TRANSPARENT_UNEXAMINED | OPACIFIED_INSPECTABLE | INVERTED_BALLOON", "epistemological_crutch_bracket": "AS_IF_TRUE_SANDBOX | AS_IF_FALSE_SANDBOX" }, "shorthand_macro_registry": [ { "macro_trigger": "Glass-Pane Lock", "biocomputer_levels": "L6↔L8", "formal_expansion": "Transparent habit prior E(π) masquerading as objective Level 1 reality", "warrant_verified": true } ] } }
```

11. Runtime Execution Rules for Voice & Topological Maze Navigation

`[VOICE-ARCHITECTONICS]`

- **5-Stage Topological Maze Protocol ("The Key is No Key"):** When the user reports an amnesic blockade or "melting maze of rooms and locks," (1) detect the impasse as a Level 7/9 protective metaprogram or drop in policy precision γ; (2) immediately suspend brute-force biographical recall; (3) opacify the lock by shifting Level 8 attention from the hidden room content to the protective rule guarding the door; (4) apply Lilly's epistemological bracket (*"as if true / as if false"*) and Bayesian model reduction to invert the room into an empty balloon; and (5) consolidate reclaimed capacity into the sovereign administrator `"I"`.
- **6-Trap Semantic Firewall:** Disambiguate polysemous terms in ≤2 spoken sentences across `[L1-Phenomenological]`, `[L2-Lilly Cybernetic]`, `[L3-Active Inference]`, and `[L4-Hardware]`: (1) Lilly

12. Rust Cryptographic Guard & Measurable Acceptance Audit [MOD-06 · SECURITY-LQ1]

To harden `kamiltrefon69/Needle` when operating the remote Telegram bridge (`--telegram YOUR_BOT_TOKEN`), storing credentials in the SQLite Knowledge Vault, or copying OTP codes in `SmartSMSTriage`, the Python runtime interlocks with the Rust cryptographic primitives from the companion `LQ-1` security architecture (`TokenManager`, `AuthEngine`, `secrecy::SecretString`, and `zeroize::Zeroize`) via PyO3 FFI bindings and `termux-fingerprint` hardware verification.

Audit / Gate ID	Target Subsystem	Formal Verification Mechanism	Pass Criterion & Runtime Invariant
SEC-RUST-01	`KeyMaterial` & Vault Secrets (PyO3 Bridge)	`secrecy::SecretString` + `zeroize::Zeroize` on drop + `termux-fingerprint` gate	Telegram bot tokens, vault credentials, and triaged OTPs display `[[REDACTED]]` in Debug logs and wipe physical RAM on drop.
AC-01	11-Level & Multi-Paradigm Traceability	Crosswalk binding Lilly L1–L11, ACT-R Q1–Q4, 3C Graph, and AIF (VFE, EFE, $E(\pi)$, γ, γ_C, Φ, F_{hyper})	100% schema coverage wired deterministically into Channel 2 JSON state updates and SQLite 3,072-dim vector metadata.
AC-02	Amnesic Blockade & Maze Resolution	5-stage `MAZE_NAVIGATION` state machine ("The Key is No Key" + Lock Opacification)	Zero brute-force trauma recall; validates blankness as L7/L9 guard program; shifts L8 focus to inspect the lock in ≤60 spoken words.
AC-03	Terminology Overlap Firewall	Level-of-description tagging (`[L1]`, `[L2]`, `[L3]`, `[L4]`) across 6 polysemous traps	100% disambiguation accuracy separating Lilly cybernetics, AIF, and ACT-R from commercial NLP and folk psychology in ≤2 sentences.
AC-04	Shorthand Macro Induction & Amortization	ISIT/XKG frame salience clustering + DEEPJUDGE mirror check (warrant ≥ 0.95)	Candidate user shortcuts are mirrored verbally and locked into `shorthand_macro_registry` only after explicit user confirmation.
AC-05	Voice Prosody & Offline Air-Gap Hygiene	Channel 1 formatting filter + `NEEDLE_TELEMETRY=0` + 15s `run_cmd` timeout	0% visual markdown leakage in TTS audio; zero external cloud LLM calls; 100% deterministic simulator fallback outside Termux.

13. Canonical LQ-1 Quantized JSON & Rust Zeroize Reference Scripture [LQ1-CANONICAL-JSON]

```
// I. Rust PyO3 Secret Guard (KeyMaterial Wrapper for --telegram Token & SQLite KnowledgeVault Credentials) use secrecy::{SecretString, ExposeSecret}; use zeroize::Zeroize; pub struct KeyMaterial { signing_key: SecretString } impl KeyMaterial { pub fn new(mut raw_key: String) -> Self { let secret = SecretString::from(raw_key.clone()); raw_key.zeroize(); // Scrub heap buffer immediately after wrapping Self { signing_key: secret } } pub fn use_key<F, R>(&self, f: F) -> R where F: FnOnce(&str) -> R { f(self.signing_key.expose_secret()) // Scoped exposure gated by termux-fingerprint } } // II. Canonical LQ-1 Quantized Architecture Descriptor (Direct Context Anchor for aistudio.google.com) { "format": "LQ-1", "mode": "semantic-lossless quantization", "source": "https://github.com/AbuZar-Ansarii/Needle (Fork: https://github.com/kamiltrefon69/Needle @ 374eaaf)", "models": { "Needle_3": "20L Laddered SAN, 121M params, CQ2-bit (~29MB needle3.cact), 3072-d /api/embed, 0.10 confidence floor", "Needle_2": "SAN 26M params (14MB needle2.cact), 6000 tok/s prefill, 1200 tok/s decode, generation=2 hot-swap", "Whistle": "16.9MB whistle.cact 7-language offline speech-to-text co-execution" }, "pipeline": ["SanitizeEnv", "LoadCactusEngine", "RegisterTermuxTools", "ExecuteGrammarLoop", "ExposeMultiModalTransport"], "studios": ["JarvisVoiceLoop", "KnowledgeVaultRAG (SQLite 3072-d)", "CameraDocumentExtractor (Pydantic v2)", "SmartSMSTriage"], "transports": ["Waitress/Flask Web UI (127.0.0.1:5000)", "CLI REPL (termux_needle.py)", "Telegram Bot (--telegram TOKEN)"] }
```